

LARAVEL

Gestion du routage

Routage de base



Routage de base

Les routes Laravel les plus basiques acceptent un URI et une fermeture, fournissant une méthode très simple et expressive de définition des routes et du comportement sans fichiers de configuration de routage compliqués :

```
use Illuminate\Support\Facades\Route;
Route::get('/greeting', function () {
    return 'Hello World';
});
```

Les fichiers de routage par défaut

Toutes les routes Laravel sont définies dans vos fichiers de route, qui se trouvent dans le répertoire **routes**.

Le fichier **routes/web.php** définit les itinéraires qui sont destinés à votre interface Web.

Les fichiers de routage par défaut

Pour la plupart des applications, vous commencerez par définir des routes dans votre fichier **routes/web.php**.

```
use Illuminate\Support\Facades\Route;
Route::get('/greeting', function () {
    return 'Hello World';
});
```

Méthodes de routeur disponibles

Le routeur vous permet d'enregistrer des routes qui répondent à n'importe quel verbe HTTP :

```
Route::get($uri, $callback);  
Route::post($uri, $callback);  
Route::put($uri, $callback);  
Route::patch($uri, $callback);  
Route::delete($uri, $callback);  
Route::options($uri, $callback);
```

Méthodes de routeur disponibles

Parfois, vous devrez peut-être enregistrer une route qui répond à plusieurs verbes HTTP. Vous pouvez le faire en utilisant la méthode `match`. Ou, vous pouvez même enregistrer une route qui répond à tous les verbes HTTP en utilisant la méthode `any` :

```
Route::match(['get', 'post'], '/', function () {  
    //  
});  
Route::any('/', function () {  
    //  
});
```

Injection de dépendance

Vous pouvez indiquer toutes les dépendances requises par votre itinéraire dans la signature de rappel de votre itinéraire. Les dépendances déclarées seront automatiquement résolues et injectées dans le rappel par le conteneur de service Laravel .

Injection de dépendance

Par exemple, vous pouvez donner un indice de type à la classe `Illuminate\Http\Request` pour que la requête HTTP actuelle soit automatiquement injectée dans votre rappel de route :

```
use Illuminate\Http\Request;
Route::get('/users', function (Request $request) {
    // ...
});
```

Redirection

Si vous définissez une route qui redirige vers un autre URI, vous pouvez utiliser la méthode **Route::redirect**. Cette méthode fournit un raccourci pratique pour que vous n'ayez pas à définir une route ou un contrôleur complet pour effectuer une simple redirection :

```
Route::redirect('/ici', '/là-bas');
```

Redirection

Par défaut, **Route::redirect** renvoie un code d'état **302**. Vous pouvez personnaliser le code d'état à l'aide du troisième paramètre facultatif :

```
Route::redirect('/ici', '/là-bas', 301);
```

Redirection

Ou, vous pouvez utiliser la méthode

Route::permanentRedirect pour renvoyer un code d'état **301**:

```
Route::permanentRedirect ('/ici', '/là-bas');
```

Afficher les routes

Si votre itinéraire ne doit renvoyer qu'une vue , vous pouvez utiliser la méthode `Route::view`. Comme la méthode `redirect`, cette méthode fournit un raccourci simple pour que vous n'ayez pas à définir une route ou un contrôleur complet.

```
Route::view('/welcome', 'welcome');
```

Afficher les routes

La méthode `view` accepte un URI comme premier argument et un nom de vue comme second argument. De plus, vous pouvez fournir un tableau de données à transmettre à la vue en tant que troisième argument facultatif :

```
Route::view('/welcome', 'welcome', ['name' => 'Ahmed']);
```

La liste des routes

La commande **Artisan route:list** peut facilement fournir une vue d'ensemble de toutes les routes définies par votre application :

```
php artisan route:list
```

La liste des routes

Par défaut, le middleware de route affecté à chaque route ne sera pas affiché dans la sortie `route:list` ; cependant, vous pouvez demander à Laravel d'afficher le middleware de route en ajoutant l'option `-v` à la commande :

```
php artisan route:list -v
```


La liste des routes

Vous pouvez également demander à Laravel de n'afficher que les routes commençant par un URI donné :

```
php artisan route:list --path=api
```

La liste des routes

De plus, vous pouvez demander à Laravel de masquer toutes les routes définies par des packages tiers en fournissant l'option **--except-vendor** lors de l'exécution de la commande `route:list` :

```
php artisan route:list --except-vendor
```

La liste des routes

De même, vous pouvez également demander à Laravel de n'afficher que les routes définies par des packages tiers en fournissant l'option **--only-vendor** lors de l'exécution de la commande `route:list` :

```
php artisan route:list --only-vendor
```

Paramètres de routage



Paramètres requis

Vous pouvez définir autant de paramètres de route que requis par votre route :

```
Route::get('/posts/{post}/comments/{comment}', function  
($postId, $commentId) {  
  //  
});
```

Paramètres requis

Les paramètres de route sont toujours entourés d'accolades {} et doivent être composés de caractères alphabétiques. Les traits de soulignement (_) sont également acceptables dans les noms de paramètre de route. Les paramètres de route sont injectés dans les rappels/contrôleurs de route en fonction de leur ordre - les noms des arguments de rappel/contrôleur de route n'ont pas d'importance.

Paramètres et injection de dépendance

Si votre route a des dépendances que vous aimeriez que le conteneur de service Laravel injecte automatiquement dans le rappel de votre route, vous devez lister vos paramètres de route après vos dépendances.

```
use Illuminate\Http\Request;
Route::get('/user/{id}', function (Request $request, $id) {
    return 'User '.$id;
});
```

Paramètres facultatifs

Parfois, vous devrez peut-être spécifier un paramètre de route qui n'est pas toujours présent dans l'URI. Vous pouvez le faire en plaçant une ? marque après le nom du paramètre. Assurez-vous de donner une valeur par défaut à la variable correspondante de la route :

```
Route::get('/user/{name?}', function ($name = null) {  
    return $name;  
});
```


Routes nommées



Routes nommées

Les routes nommées permettent la génération pratique d'URL ou de redirections pour des routes spécifiques. Vous pouvez spécifier un nom pour une route en enchaînant la méthode **name** sur la définition de route :

```
Route::get('/user/profile', function () {  
    //  
})->name('profile');
```

Routes nommées

Vous pouvez également spécifier des noms de route pour les actions du contrôleur :

```
Route::get(
  '/user/profile',
  [UserProfileController::class, 'show']
)->name('profile');
```

Génération d'URL vers des routes nommées

Une fois que vous avez attribué un nom à une route donnée, vous pouvez utiliser le nom de la route lors de la génération d'URL ou de redirections via les fonctions de Laravel `route` et d'assistance `redirect` :

```
// Generating URLs...  
$url = route('profile');  
// Generating Redirects...  
return redirect()->route('profile');
```

Génération d'URL vers des routes nommées

Si la route nommée définit des paramètres, vous pouvez passer les paramètres comme deuxième argument à la fonction route. Les paramètres donnés seront automatiquement insérés dans l'URL générée dans leurs positions correctes :

```
Route::get('/user/{id}/profile', function ($id) {  
    //  
})->name('profile');  
$url = route('profile', ['id' => 1]);
```

Génération d'URL vers des routes nommées

Si vous transmettez des paramètres supplémentaires dans le tableau, ces paires clé/valeur seront automatiquement ajoutées à la chaîne de requête de l'URL générée :

```
Route::get('/user/{id}/profile', function ($id) {  
    //  
    }->name('profile');  
    $url = route('profile', ['id' => 1, 'photos' => 'yes']);  
    // /user/1/profile?photos=yes
```

Groupe de routage



La liste des routes

Les routes peuvent être regroupées pour éviter la répétition du code.

- Les groupes de routage vous permettent de partager des attributs de routage, tels que le middleware, sur un grand nombre de routages sans avoir à définir ces attributs sur chaque routage individuel.
- Disons que tous les URI avec un préfixe de /admin utilisent un certain middleware appelé admin et qu'ils vivent tous dans l'espace de noms `App\Http\Controllers\Admin`.

La liste des routes

Une manière propre de représenter cela en utilisant des groupes de routage est la suivante:

```
Route::group(['namespace' => 'Admin',  
             'middleware' => 'admin',  
             'prefix' => 'admin'], function () {  
    // something.dev/admin  
    // 'App\Http\Controllers\Admin\IndexController'  
    // Uses admin middleware  
Route::get('/', ['uses' => 'IndexController@index']);  
    // something.dev/admin/logs  
    // 'App\Http\Controllers\Admin\LogsController'  
    // Uses admin middleware  
Route::get('/logs', ['uses' => 'LogsController@index']);  
});
```

Liaisons de modèles de routes



Liaison Implicite

Laravel résout automatiquement les modèles Eloquent définis dans les itinéraires ou les actions de contrôleur dont les noms de variable correspondent à un nom de segment de route.

Par exemple:

```
Route::get('api/users/{user}', function (App\User $user) {  
    return $user->email;  
});
```

Liaison Implicite

Dans cet exemple, comme la variable utilisateur éloquent \$ définie sur la route correspond au segment {utilisateur} dans l'URI de la route, Laravel injectera automatiquement l'instance de modèle dont l'ID correspond à la valeur correspondante de l'URI de la demande. Si une instance de modèle correspondante n'est pas trouvée dans la base de données, une réponse HTTP 404 sera automatiquement générée.

Si le nom de la table du modèle est composé de plusieurs mots, pour que la liaison de modèle implicite fonctionne, la variable d'entrée doit être en minuscules;

Liaison Implicite

Par exemple, si l'utilisateur peut effectuer une action quelconque et que nous voulons accéder à cette action, l'itinéraire sera le suivant:

```
Route::get('api/useractions/{useraction}', function  
(App\UserAction $useraction) {  
    return $useraction->description;  
});
```

Liaison Explicite

Pour enregistrer une liaison explicite, utilisez la méthode du modèle du routeur pour spécifier la classe d'un paramètre donné. Vous devez définir vos liaisons de modèle explicites dans la méthode de démarrage de la classe **RouteServiceProvider**

```
public function boot()  
{  
    parent::boot();  
    Route::model('user', App\User::class);  
}
```

Liaison Explicite

Ensuite, nous pouvons définir une route contenant le paramètre {utilisateur}.

```
public function boot()  
{  
    parent::boot();  
    Route::model('user', App\User::class);  
}  
$router->get('profile/{user}', function(App\User $user) {  
    });
```

Liaison Explicite

Comme nous avons lié tous **{user}** paramètres **{user}** au modèle **App\User**, une instance User sera injectée dans la route.

Ainsi, par exemple, une demande de `profile/1` injectera l'instance d'utilisateur de la base de données dont l'identifiant est 1.

Si une instance de modèle correspondante n'est pas trouvée dans la base de données, une réponse HTTP 404 sera automatiquement générée

Routes de repli



Routes de repli

En utilisant la méthode `Route::fallback`, vous pouvez définir une route qui sera exécutée lorsqu'aucune autre route ne correspond à la demande entrante.

Routes de repli

```
Route::fallback(function ()  
{  
    //  
});
```

En règle générale, les requêtes non gérées afficheront automatiquement une page "404" via le gestionnaire d'exceptions de votre application. Cependant, comme vous définissez généralement la fallbackroute dans votre fichier routes/web.php, tous les middleware du groupe middleware web s'appliqueront à la route. Vous êtes libre d'ajouter un middleware supplémentaire à cette route si nécessaire :

(spoofing) de la méthode du formulaire

La liste des routes

Les formulaires HTML ne prennent pas en charge les actions **PUT**, **PATCH** ou **DELETE**. Ainsi, lors de la définition de routes appelées **PUT**, **PATCH** ou **DELETE** à partir d'un formulaire HTML, vous devrez ajouter un champ masqué `_method` au formulaire. La valeur envoyée avec le champ `_method` sera utilisée comme méthode de requête HTTP :

```
<form action="/example" method="POST">  
<input type="hidden" name="_method" value="PUT">  
<input type="hidden" name="_token" value="{{ csrf_token() }}">  
</form>
```

La liste des routes

Pour plus de commodité, vous pouvez utiliser la directive Blade `@method` pour générer le champ `_method` de saisie :

```
<form action="/example" method="POST">  
@method( 'PUT' )  
@csrf  
</form>
```

Accès à la route courante



Accès à la route courante

Vous pouvez utiliser les méthodes `current`, **`currentRouteName`** et **`currentRouteAction`** sur la façade `Route` pour accéder aux informations sur la route traitant la requête entrante :

```
use Illuminate\Support\Facades\Route;  
$route = Route::current(); // Illuminate\Routing\Route  
$name = Route::currentRouteName(); // string  
$action = Route::currentRouteAction(); // string
```


Cross Origin Resource Sharing (CORS)

Laravel peut répondre automatiquement aux requêtes HTTP CORS OPTIONS avec des valeurs que vous configurez. Tous les paramètres CORS peuvent être configurés dans le fichier `config/cors.php` de configuration de votre application. Les requêtes OPTIONS seront automatiquement traitées par le middleware `HandleCors` qui est inclus par défaut dans votre pile middleware globale. Votre pile middleware globale se trouve dans le noyau HTTP de votre application (`App\Http\Kernel`).

Mise en cache des routes

Lors du déploiement de votre application en production, vous devez tirer parti du cache de routage de Laravel.

L'utilisation du cache de route réduira considérablement le temps nécessaire pour enregistrer toutes les routes de votre application.

Mise en cache des routes

Pour générer un cache de route,
exécutez la commande Artisan
route:cache :

```
php artisan route:cache
```

Mise en cache des routes

Après avoir exécuté cette commande, votre fichier de routes en cache sera chargé à chaque requête. N'oubliez pas que si vous ajoutez de nouvelles routes, vous devrez générer un nouveau cache de routes. Pour cette raison, vous ne devez exécuter la commande **route:cache** que pendant le déploiement de votre projet.

Mise en cache des routes

Vous pouvez utiliser la commande **route:clear** pour vider le cache de route:

```
php artisan route:clear
```